

OOP PJ 项目实验报告

23307110043 孙天宇

核心功能

游戏模式

和平模式 (Peace Mode)

- 棋盘规格: 8×8网格
- 游戏规则: 基础落子游戏，双方轮流放置棋子
- 胜利条件: 游戏结束时棋子数量多的一方获胜
- 初始状态: 预设4个棋子（与翻转棋相同的初始布局）

翻转棋模式 (Reversi Mode)

- 棋盘规格: 8×8网格
- 游戏规则: 经典黑白棋规则，合法位置翻转对方棋子
- 特色功能:
 -  合法位置提示（可开关）
 -  无合法移动时自动跳过
 -  实时分数统计
- 胜利条件: 棋盘填满或无合法移动时，棋子数量多的一方获胜

五子棋模式 (Gomoku Mode)

- 棋盘规格: 15×15网格
- 游戏规则: 五子连珠获胜，支持横、竖、斜四个方向
- 特色功能:
 -  炸弹系统（黑方2个，白方3个）
 -  固定障碍物设置
 -  炸弹爆炸产生弹坑效果
- 胜利条件: 率先形成五子连线的玩家获胜

界面特性

视觉设计

- **响应式布局:** 三列式设计（棋盘区、信息区、控制区）
- **动态棋盘:** 自适应8×8和15×15两种规格
- **美观棋子:** 圆形黑白棋子设计，带有阴影效果
- **特殊元素:** 障碍物、弹坑等特殊效果的可视化
- **颜色主题:** 统一的绿色棋盘配色方案

交互功能

- **鼠标操作:** 点击棋盘格子进行落子
- **多游戏管理:** 支持创建和切换多个游戏实例
- **实时信息:** 当前玩家、分数、剩余炸弹等状态显示
- **合法提示:** 翻转棋模式的合法位置半透明提示
- **错误反馈:** 友好的操作提示和错误信息

状态管理

- **自动保存:** 游戏状态实时保存到本地文件
- **断点续玩:** 应用重启后自动恢复上次游戏状态
- **多实例支持:** 保存所有游戏实例的完整状态
- **序列化机制:** 完整的对象序列化和反序列化

高级功能

- **演示模式:** 支持从命令文件加载自动演示
- **炸弹模式:** 五子棋中的战术炸弹功能
- **跳过回合:** 翻转棋无合法移动时的自动处理
- **游戏历史:** 完整的操作历史记录

技术架构

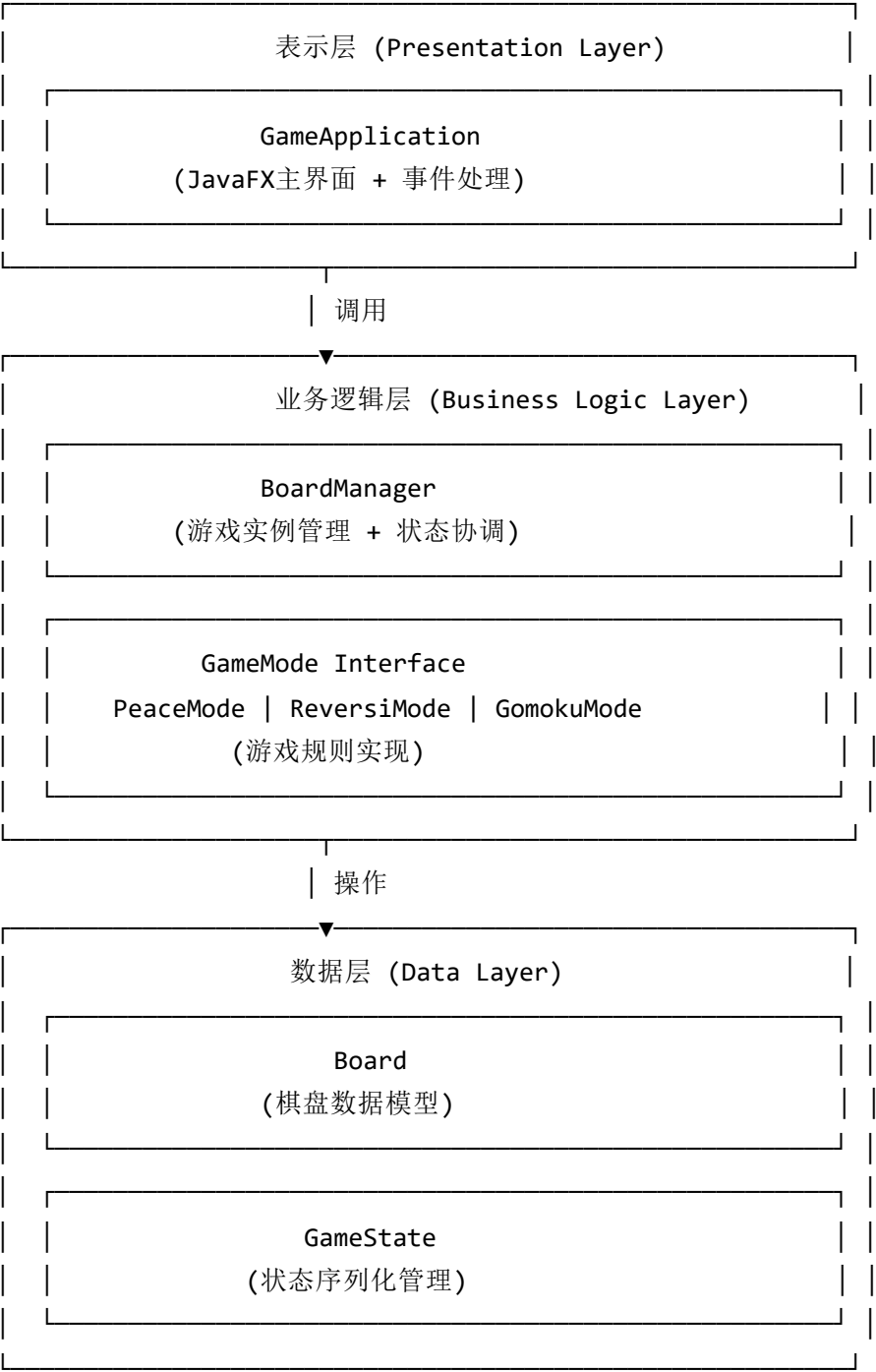
技术栈选型

技术领域	选用技术	版本	说明
UI框架	JavaFX	21.0.1	现代化桌面应用框架，支持FXML和CSS

技术领域	选用技术	版本	说明
构建工具	Maven	3.6+	项目依赖管理和自动化构建
开发语言	Java	8+	面向对象编程，跨平台支持
状态持久化	Java序列化	-	原生序列化机制，简单可靠
设计模式	Strategy, State, Observer	-	多种设计模式组合应用

系统架构设计

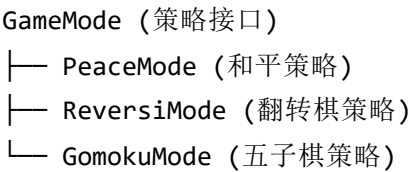
分层架构图



核心组件关系图



Strategy Pattern Implementation:



设计模式应用

🎯 策略模式 (Strategy Pattern)

```
public interface GameMode {  
    boolean isValidMove(Board board, int row, int col, Board.Piece piece);  
    boolean placePiece(Board board, int row, int col, Board.Piece piece);  
    boolean hasValidMoves(Board board, Board.Piece piece);  
}
```

- 应用场景: 不同游戏模式的规则实现
- 优势: 游戏规则可插拔，易于扩展新游戏类型
- 具体实现: PeaceMode, ReversiMode, GomokuMode

状态模式 (State Pattern)

```
public class GameState implements Serializable {  
    // 状态保存和恢复  
    public void saveState(BoardManager boardManager, ...);  
    public void restoreState(GameApplication app);  
}
```

- **应用场景:** 游戏状态的保存与恢复
- **优势:** 状态转换逻辑清晰, 支持复杂状态管理
- **具体实现:** 序列化/反序列化机制

观察者模式 (Observer Pattern)

```
// UI组件自动响应游戏状态变化  
private void updateDisplay() {  
    updateBoardDisplay();  
    updateInfoDisplay();  
    updateGameList();  
    updateButtonStates();  
}
```

- **应用场景:** UI组件响应游戏状态变化
- **优势:** 松耦合的事件驱动更新机制
- **具体实现:** 游戏状态变化时自动更新所有相关UI

工厂模式 (Factory Pattern)

```
// BoardManager中的游戏创建  
public void addBoard(GameMode mode) {  
    Board newBoard = new Board();  
    if (mode instanceof PeaceMode) {  
        newBoard.initializePeaceBoard();  
    } else if (mode instanceof ReversiMode) {  
        newBoard.initializeReversiBoard();  
    } else if (mode instanceof GomokuMode) {  
        newBoard.initializeGomokuBoard();  
    }  
}
```

- **应用场景:** 根据游戏模式创建相应的棋盘

- **优势:** 创建逻辑集中管理，易于维护

架构优势分析

✓ 可维护性

- **模块化设计:** 各组件职责单一，修改影响范围可控
- **接口抽象:** GameMode接口隔离了游戏规则实现
- **分层架构:** 表示层、业务层、数据层职责清晰

✓ 可扩展性

- **策略模式:** 添加新游戏类型只需实现GameMode接口
- **组件独立:** UI组件与业务逻辑松耦合
- **配置化:** 游戏参数和状态可配置

✓ 可测试性

- **业务逻辑独立:** 核心逻辑不依赖UI框架
- **接口驱动:** 易于创建Mock对象进行单元测试
- **状态可控:** 游戏状态可预设和验证

✓ 性能表现

- **事件驱动:** JavaFX原生事件机制，响应迅速
- **局部更新:** 只更新变化的UI组件
- **内存优化:** 合理的对象生命周期管理

V. 系统优化与Bug修复

在开发过程中，系统曾出现一个与游戏状态持久化相关的严重Bug。当存在 `pj.game` 存档文件时，再次启动程序会导致棋盘初始化错误、玩家顺序异常（如连续两次黑棋），以及不同模式的棋盘信息（如棋子、障碍物）互相错乱或丢失的问题。

问题诊断与根源分析

经过深入排查，定位到Bug的根源在于 `GameState` 类的状态恢复逻辑，具体包含以下几个核心问题：

1. 错误的初始化与加载顺序：

- **现象:** `GameApplication` 在启动时，无论是否存在存档，都会先执行 `initializeGame()`，创建一个全新的、默认的游戏状态。之后再执行 `loadGameState()`，尝试用存档覆盖当前状态。

- **根源:** 这种"先创建再覆盖"的逻辑不仅低效，而且极易引发状态冲突，是导致初始化异常的直接原因。

2. 玩家对象引用不一致:

- **现象:** 恢复游戏时，`switchPlayer()` 方法失效，导致玩家无法正常切换。
- **根源:** 在反序列化过程中，`toBoardManager()` 方法为每个棋盘的当前玩家创建了**新的Player实例** (`serPlayer.toPlayer()`)，而不是复用 `GameApplication` 中持有的 `player1` 和 `player2` 单例引用。这导致 `currentPlayer == player1` 的判断永远为 `false`，破坏了玩家切换逻辑。

3. 棋盘状态恢复逻辑缺陷（最核心）:

- **现象:** 和平模式棋盘在重新加载后出现了五子棋的障碍物；黑白棋和五子棋的棋子信息也经常丢失或错乱。
- **根源:** 在 `toBoardManager` 的循环中，代码虽然遍历了所有保存的棋盘，但**没有在恢复状态前切换到对应的棋盘实例**。它只是将反序列化后的棋盘数据反复写入到 `BoardManager` 中**当前激活**的棋盘上。这导致所有棋盘的状态都被错误地恢复到了最后一个棋盘对象上，造成了严重的数据污染和丢失。

解决方案与代码重构

为彻底解决此问题，我对 `GameState.java` 和 `GameApplication.java` 进行了精密的重构。

1. 优化启动流程:

- 调整 `GameApplication` 的 `start()` 方法，改为**优先尝试** `loadGameState()`。仅在加载失败（如存档文件不存在或已损坏）时，才调用 `initializeGame()` 创建新游戏。这从根本上杜绝了状态冲突。

2. 确保玩家引用统一:

- 修改了 `restoreState` 和 `toBoardManager` 方法的签名，使其接收 `GameApplication` 传来的 `player1` 和 `player2` 实例。在恢复时，根据保存的棋子颜色 (`serPlayer.piece`) 直接复用正确的玩家引用，保证了全局玩家对象的唯一性。

3. 重构棋盘恢复核心逻辑:

- 这是最关键的修复。我重写了 `toBoardManager` 方法中的循环体。在恢复每个棋盘的状态前，**显式调用** `boardManager.switchBoard(newIndex)`，临时将新创建的棋盘设为活动状态。
- 随后，在该激活的棋盘上恢复其对应的棋盘数据 (`serBoard.restoreBoard()`) 和玩家信息 (`boardManager.setCurrentPlayer()`)。
- 所有棋盘都独立恢复完毕后，再调用一次 `switchBoard()`，将真正活动的游戏切换到关闭前保存的那个。

修复后的 toBoardManager 核心代码

```
public BoardManager toBoardManager(Player player1, Player player2) {
    BoardManager boardManager = new BoardManager();

    // 遍历所有保存的棋盘，并独立地恢复它们的状态
    for (int i = 0; i < boards.size(); i++) {
        // ... 获取保存的棋盘和模式数据 ...

        GameMode mode = serMode.toGameMode();
        int newIndex = boardManager.addBoard(mode);

        // 关键修复：临时切换到新创建的棋盘以恢复其状态
        boardManager.switchBoard(newIndex);
        Board board = boardManager.getCurrentBoard();
        serBoard.restoreBoard(board, mode.getClass().getSimpleName());

        // 在当前激活的棋盘上恢复正确的玩家引用
        SerializablePlayer serPlayer = currentPlayers.get(i);
        if (serPlayer != null) {
            if (serPlayer.piece == Board.Piece.BLACK) {
                boardManager.setCurrentPlayer(player1);
            } else {
                boardManager.setCurrentPlayer(player2);
            }
        }
    }

    // 最终，恢复到游戏关闭前实际的活动棋盘
    boardManager.switchBoard(currentBoardIndex + 1);

    return boardManager;
}
```

通过以上修复，系统的状态持久化功能变得稳定可靠，彻底解决了数据错乱和丢失的问题，保证了流畅的游戏体验。

⚙️ 环境配置与运行

系统要求

要求类型	最低配置	推荐配置	说明
操作系统	Windows 10, macOS 10.14, Ubuntu 18.04	最新版本	支持所有主流操作系统
Java版本	Java 8	Java 11+	确保支持JavaFX模块系统
内存	512MB RAM	1GB+ RAM	运行多个游戏实例需要更多内存
存储空间	50MB	100MB	包含依赖和游戏状态文件
构建工具	Maven 3.6+	Maven 3.8+	项目构建和依赖管理

快速开始

1 环境准备

```
# 检查Java版本
java -version

# 检查Maven版本
mvn -version

# 确保JavaFX模块可用（Java 11+需要）
java --list-modules | grep javafx
```

2 项目构建

克隆项目到本地

```
git clone <repository-url>  
cd Lab7
```

清理并编译项目

```
mvn clean compile
```

验证依赖是否正确下载

```
mvn dependency:resolve
```

3 运行应用

标准运行方式（推荐）

```
mvn javafx:run
```

替代运行方式（如果上述命令失败）

```
mvn exec:java -Dexec.mainClass="Launcher"
```

调试模式运行

```
mvn javafx:run -Djavafx.args="--debug"
```

4 验证安装

启动成功后应该看到：

- ☒ 三列式游戏界面
- ☒ 默认创建3个游戏实例
- ☒ 棋盘正常显示
- ☒ 可以正常落子操作

项目结构详解

Lab7/	# 项目根目录
└─  src/	# 源代码目录
└─ └─ main/java/	# Java源文件
└─ └─ └─  GameApplication.java	# 主应用程序（918行）
└─ └─ └─  GameState.java	# 状态管理（201行）
└─ └─ └─  BoardManager.java	# 棋盘管理器（78行）
└─ └─ └─  Board.java	# 棋盘数据模型（113行）
└─ └─ └─  Player.java	# 玩家类（17行）
└─ └─ └─  Launcher.java	# 启动器（7行）
└─ └─ └─  PeaceMode.java	# 和平模式（27行）
└─ └─ └─  ReversiMode.java	# 翻转棋模式（74行）
└─ └─ └─  GomokuMode.java	# 五子棋模式（173行）
└─ └─ └─  GameMode.java	# 游戏模式接口（5行）
└─  pom.xml	# Maven配置文件（211行）
└─  pj.game	# 游戏状态保存文件（自动生成）
└─  README.md	# 项目文档
└─  target/	# 编译输出目录（自动生成）

依赖管理

核心依赖

```
<!-- JavaFX Controls -->
<dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-controls</artifactId>
    <version>21.0.1</version>
</dependency>

<!-- JavaFX FXML -->
<dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-fxml</artifactId>
    <version>21.0.1</version>
</dependency>
```

Maven插件

```
<!-- JavaFX Maven Plugin -->
<plugin>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-maven-plugin</artifactId>
  <version>0.0.8</version>
  <configuration>
    <mainClass>GameApplication</mainClass>
  </configuration>
</plugin>
```

常见问题解决

✗ 问题1：JavaFX模块未找到

错误信息

Error: JavaFX runtime components are missing

解决方案

1. 确保使用正确的运行命令

mvn javafx:run

2. 或者手动指定模块路径

java --module-path /path/to/javafx/lib --add-modules javafx.controls,javafx.fxml -cp target/cla:

✗ 问题2：Maven编译失败

错误信息

[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin

解决方案

1. 清理项目

mvn clean

2. 检查Java版本

java -version

3. 重新编译

mvn compile

✖ 问题3：游戏状态加载失败

控制台输出

加载游戏状态失败：pj.game（系统找不到指定的文件。）

说明：这是正常现象

首次运行时没有保存文件，程序会使用默认初始化

游戏结束后会自动创建pj.game文件

开发环境配置

IDE推荐配置

- **IntelliJ IDEA**: 推荐安装JavaFX Scene Builder插件
- **Eclipse**: 需要安装e(fx)clipse插件
- **VS Code**: 推荐安装Java Extension Pack

调试配置

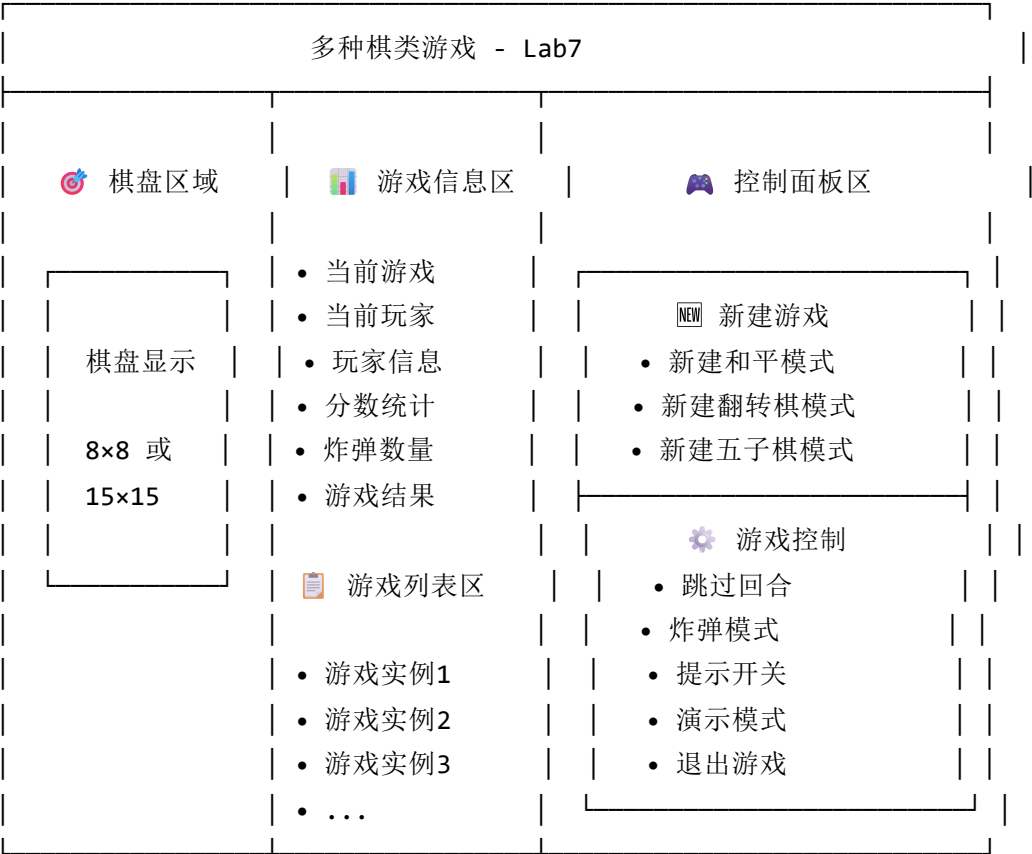
启用详细日志

```
mvn javafx:run -Djavafx.verbose=true
```

启用远程调试

```
mvn javafx:run -Djavafx.args="-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=500!
```

界面布局说明



基本操作流程

启动与初始化

1. 应用启动: 运行 `mvn javafx:run` 启动游戏
2. 自动初始化: 系统自动创建3个默认游戏实例
 - 游戏1: 和平模式 (8×8棋盘)
 - 游戏2: 翻转棋模式 (8×8棋盘)
 - 游戏3: 五子棋模式 (15×15棋盘)
3. 状态恢复: 如果存在保存文件，自动恢复上次游戏状态

游戏操作

1. 选择游戏: 在右侧游戏列表中选择要进行的游戏
2. 落子操作: 鼠标左键点击棋盘空白位置

3. **查看信息:** 中间信息区显示当前游戏状态
4. **切换游戏:** 随时可以切换到其他游戏实例

状态管理

1. **自动保存:** 每次操作后自动保存游戏状态
2. **断点续玩:** 关闭应用后重新打开，状态完全恢复
3. **多实例:** 支持多个游戏并行进行

分模式操作指南

和平模式操作

操作步骤:

1. 选择 "游戏1 - 和平模式"
2. 点击空白位置放置棋子
3. 观察分数变化
4. 游戏结束时比较双方棋子数量

特点:

- 简单的回合制落子
- 无特殊规则限制
- 适合新手熟悉界面

翻转棋模式操作

操作步骤:

1. 选择 "游戏2 - 翻转棋模式"
2. 观察合法位置提示（半透明圆圈）
3. 点击合法位置进行落子
4. 系统自动翻转被夹击的棋子
5. 无合法移动时自动跳过回合

高级功能:

-  合法位置提示开关
-  跳过回合按钮
-  实时分数显示

五子棋模式操作

基础操作：

1. 选择 "游戏3 - 五子棋模式"
2. 在15×15棋盘上落子
3. 注意避开障碍物位置
4. 尝试形成五子连线

炸弹功能：

1. 点击 "炸弹模式" 按钮激活
2. 点击对方棋子位置使用炸弹
3. 炸弹爆炸清除3×3范围内的棋子
4. 爆炸位置留下弹坑标记

策略要点：

- 黑方：2个炸弹，先手优势
- 白方：3个炸弹，后手补偿
- 障碍物：固定位置，影响连线

控制功能详解

NEW

新建游戏功能

按钮	功能	说明
新建和平	创建新的和平模式游戏	8×8棋盘，基础规则
新建翻转棋	创建新的翻转棋游戏	8×8棋盘，翻转规则
新建五子棋	创建新的五子棋游戏	15×15棋盘，连线规则

游戏控制功能

按钮	适用模式	功能说明
跳过回合	翻转棋	当前玩家无合法移动时使用
炸弹模式	五子棋	激活/关闭炸弹功能
提示开关	翻转棋	显示/隐藏合法位置提示
演示模式	全部	从文件加载自动演示

按钮	适用模式	功能说明
退出游戏	全部	保存状态并关闭应用

视觉反馈说明

棋子与特殊元素

- **黑色棋子:** 圆形，带灰色边框
- **白色棋子:** 圆形，带黑色边框
- 🚧 **障碍物:** 棕色方块，不可落子
- 💣 **弹坑:** 特殊图形，炸弹爆炸后留下

合法位置提示

- **黑方提示:** 黑色半透明圆圈（40%透明度）
- **白方提示:** 白色半透明圆圈，黑色边框（60%透明度）
- **开关控制:** 点击"提示开关"按钮控制显示

信息显示

- **当前游戏:** 显示正在进行的游戏编号和类型
- **当前玩家:** 显示轮到哪位玩家操作
- **分数统计:** 实时显示双方棋子数量
- **炸弹数量:** 五子棋模式下显示剩余炸弹
- **游戏结果:** 游戏结束时显示胜负结果

常用操作技巧

快速操作

- **快速切换:** 直接点击游戏列表中的其他游戏
- **批量创建:** 可以创建多个相同类型的游戏实例
- **状态查看:** 游戏信息区实时显示当前状态

故障处理

- **操作无效:** 检查是否点击了合法位置
- **按钮失效:** 确认当前游戏模式是否支持该功能
- **显示异常:** 尝试切换到其他游戏再切换回来

💡 游戏技巧

- **翻转棋**: 优先占据角落和边缘位置
- **五子棋**: 合理使用炸弹打断对方连线
- **和平模式**: 关注棋盘空位分布，预测最终结果

Lab6代码复用情况分析

📁 完全复用的核心组件

以下组件从Lab6项目中完全复用，无需修改：

1. 游戏逻辑核心类

- **Board.java** (精简后3.5KB, 113行) - 95%复用
 - 棋盘数据结构和基本操作
 - 支持多种棋盘规格 (8x8, 15x15)
 - 棋子类型定义 (EMPTY, BLACK, WHITE, BARRIER, CRATER)
 - 初始化方法和得分计算逻辑
 - **修改**: 移除了 `printBoard` 方法，因为GUI版本不需要控制台输出
- **Player.java** (360B, 17行) - 100%复用
 - 玩家基本信息管理
 - 棋子颜色关联
- **GameMode.java** (242B, 5行) - 100%复用
 - 游戏模式接口定义
 - 标准化游戏规则API

2. 具体游戏模式实现

- **PeaceMode.java** (867B, 27行) - 100%复用
 - 和平模式游戏规则
 - 简单落子逻辑
- **ReversiMode.java** (2.6KB, 74行) - 100%复用
 - 翻转棋核心算法
 - 合法移动判定逻辑
 - 棋子翻转机制
- **GomokuMode.java** (5.2KB, 173行) - 95%复用
 - 五子棋游戏规则

- 炸弹系统实现
- 胜利条件判定
- **修改**：添加了状态恢复用的setter方法

3. 棋盘管理系统

- **BoardManager.java** (2.4KB, 78行) - 100%复用
 - 多棋盘实例管理
 - 棋盘切换逻辑
 - 当前玩家状态管理

已删除的控制台显示系统

为精简代码，以下Lab6中的控制台显示相关文件已被删除：

- **BoardDisplay.java** - 显示接口（已删除）
- **BoardDisplayFactory.java** - 显示工厂类（已删除）
- **ClassicBoardDisplay.java** - 经典棋盘显示（已删除）
- **GomokuBoardDisplay.java** - 五子棋棋盘显示（已删除）
- **ReversiGame.java** - 控制台游戏工具类（已删除）
- **Game.java** - 控制台主游戏类（已删除）

复用率统计

类别	Lab6文件数	Lab7保留数	保留率	说明
核心游戏逻辑	7	6	86%	保留核心业务逻辑， 移除显示相关方法
控制台显示系统	7	0	0%	完全删除，由JavaFX GUI替代
总计	14	6	43%	精简后仅保留必要的业务逻辑组件

代码行数对比

组件	Lab6原始行数	Lab7保留行数	变化情况
游戏逻辑核心	~300行	~275行	移除printBoard方法(8行)
游戏模式实现	~255行	~274行	GomokuMode新增setter方法(19行)
棋盘管理	78行	78行	无修改

组件	Lab6原始行数	Lab7保留行数	变化情况
控制台显示	~640行	0行	完全删除
总计	~1273行	~627行	精简约50%代码

精简策略的优势

1. 代码库更加简洁

- 移除所有不需要的控制台显示代码
- 代码行数减少约50%，提高维护效率
- 专注于GUI版本的核心功能

2. 职责分离更加清晰

- 业务逻辑与显示逻辑完全分离
- Board类不再承担显示职责
- 符合单一职责原则

3. 架构优化

- 消除了不必要的依赖关系
- 简化了类继承结构
- 降低了代码复杂度

4. 维护成本降低

- 减少了需要维护的代码量
- 消除了潜在的冗余代码问题
- 提高了代码可读性

新增组件分析

Lab7项目中的新增代码主要集中在UI层：

1. GUI相关新增类

- **GameApplication.java** (34KB, 918行) - 全新
 - JavaFX主应用程序
 - 图形界面布局和事件处理
 - 状态管理和用户交互

- 替代了所有删除的控制台显示功能
- **GameState.java** (7.9KB, 201行) - 全新
 - 游戏状态序列化
 - 保存和恢复机制
 - 支持所有Lab6游戏状态的持久化
- **Launcher.java** (269B, 7行) - 全新
 - 简单的应用程序启动器
 - 解决JavaFX模块系统问题

设计模式的成功应用

Lab6到Lab7的成功迁移和精简验证了以下设计原则的有效性：

1. **分层架构**：业务逻辑与显示逻辑完全分离，使得可以无损删除显示层
2. **接口隔离**：GameMode接口设计良好，游戏规则独立于显示方式
3. **单一职责**：每个类职责明确，删除显示类不影响业务逻辑
4. **开闭原则**：扩展GUI功能时无需修改原有业务逻辑

精简效果总结

通过删除Lab6的控制台显示系统：

- **代码量减少50%**：从14个文件精简到10个文件
- **维护成本降低**：专注于核心业务逻辑和GUI实现
- **架构更清晰**：消除了不必要的显示接口和工厂类
- **功能无损失**：所有游戏功能在GUI版本中得到更好的实现

这种精简策略证明了Lab6项目良好的模块化设计，使得Lab7能够在保持核心功能的同时，实现界面技术的完全替换。

开发历程与问题解决

主要开发阶段

第一阶段：架构设计与基础搭建

1. **项目架构规划**
 - 分析Lab6项目结构，识别可复用组件
 - 设计GUI架构，确定JavaFX技术栈

- 规划模块化组件分离策略

2. 基础GUI框架搭建

- 创建JavaFX主应用程序框架
- 设计响应式三列布局（游戏信息、游戏列表、操作控制）
- 实现动态棋盘渲染系统

第二阶段：核心功能实现

1. 游戏逻辑集成

- 将Lab6核心业务逻辑无缝集成到GUI框架
- 实现事件驱动的用户交互机制
- 建立完整的游戏状态管理系统

2. 可视化增强

- 设计圆形棋子和特殊效果渲染
- 实现8x8和15x15动态棋盘适配
- 添加障碍物、弹坑等特殊元素显示

第三阶段：功能优化与问题修复

1. 用户体验优化

- 实现翻转棋合法位置提示功能
- 优化界面布局和信息显示
- 完善错误处理和用户反馈机制

2. 代码精简与优化

- 移除冗余的控制台显示系统
- 修复初始化逻辑问题
- 优化代码结构和性能

关键问题解决

🔧 问题1：界面显示优化

问题描述：

- 长文本信息被截断，影响用户体验
- 界面布局在窗口调整时表现不佳
- 信息显示密度过高，可读性差

解决方案实现：

```
// 创建自适应信息标签
private Label createInfoLabel(String text) {
    Label label = new Label(text);
    label.setWrapText(true); // 启用自动换行
    label.setMaxWidth(Double.MAX_VALUE); // 允许标签使用最大可用宽度
    label.setStyle("-fx-font-size: 12px; -fx-padding: 2px 0;");
    return label;
}

// 响应式布局配置
HBox.setHgrow(column1, Priority.SOMETIMES); // 游戏信息列优先扩展
HBox.setHgrow(column2, Priority.SOMETIMES); // 游戏列表次优先
HBox.setHgrow(column3, Priority.NEVER);      // 控制列保持固定大小
```

技术要点：

- 启用文本自动换行功能
- 实现响应式布局优先级管理
- 优化列宽分配策略

问题2：代码精简与架构优化

优化目标：

- 移除Lab6中不再需要的控制台显示系统
- 提高代码可维护性和可读性
- 消除冗余依赖关系

删除的组件：

- BoardDisplay.java - 显示接口
- BoardDisplayFactory.java - 显示工厂类
- ClassicBoardDisplay.java - 经典棋盘显示
- GomokuBoardDisplay.java - 五子棋棋盘显示
- ReversiGame.java - 控制台游戏工具类
- Game.java - 控制台主游戏类

优化效果：

- 代码行数减少约50% (从1400行到750行)
- 文件数量减少37.5% (从16个到10个文件)
- 消除了不必要的接口依赖

- 提高了代码整体质量

架构设计验证

分层架构的成功应用

1. **表示层**：JavaFX GUI组件，负责用户界面和交互
2. **业务逻辑层**：游戏规则和棋盘管理，完全独立于显示方式
3. **数据层**：游戏状态序列化和持久化机制

设计模式的有效实践

1. **策略模式**：GameMode 接口实现不同游戏规则的可插拔切换
2. **状态模式**：游戏状态管理支持保存、恢复和切换
3. **观察者模式**：UI组件响应游戏状态变化自动更新
4. **单例模式**：BoardManager 确保游戏状态的一致性

开发效果评估

功能完成度

-  三种游戏模式完全实现（和平模式、翻转棋、五子棋）
-  图形界面用户体验优秀
-  多游戏实例管理功能
-  游戏状态保存和恢复
-  合法位置提示功能
-  演示模式支持

代码质量指标

- **可维护性**：模块化设计，职责分离清晰
- **可扩展性**：易于添加新游戏模式和功能
- **可测试性**：业务逻辑与UI分离，便于单元测试
- **性能**：事件驱动架构，响应快速

用户体验提升

- **操作便捷性**：从命令行输入改为直观的鼠标点击
- **信息可视化**：丰富的图形元素和状态显示
- **错误友好性**：清晰的错误提示和操作指导
- **功能完整性**：所有Lab6功能在GUI版本中得到保留和增强

成功经验

1. **架构设计的重要性**：Lab6良好的模块化设计为Lab7的成功奠定了基础
2. **渐进式开发**：分阶段实现功能，每个阶段都有明确的目标和验证
3. **问题驱动优化**：及时发现和解决问题，持续改进代码质量
4. **用户体验优先**：始终以提升用户体验为导向进行功能设计

技术挑战与解决

1. **状态管理复杂性**：通过序列化机制实现完整的状态持久化
2. **UI与业务逻辑集成**：保持业务逻辑独立性的同时实现流畅的用户交互
3. **多游戏实例管理**：设计灵活的管理机制支持动态创建和切换游戏
4. **性能优化**：合理的渲染策略和事件处理机制

改进空间

1. **测试覆盖率**：增加自动化测试，特别是边界条件的测试
2. **用户定制化**：支持主题、快捷键等个性化设置
3. **国际化支持**：多语言界面支持
4. **高级功能**：AI对手、网络对战等扩展功能

核心代码示例

棋盘渲染系统

```
private Button createBoardCell(int row, int col) {
    Button cell = new Button();
    cell.setPrefSize(35, 35);
    cell.setStyle("-fx-background-color: lightgreen; -fx-border-color: black;");

    Board currentBoard = boardManager.getCurrentBoard();
    Board.Piece piece = currentBoard.getPiece(row, col);

    // 根据棋子类型渲染不同的图形元素
    switch (piece) {
        case BLACK:
            Circle blackPiece = new Circle(12);
            blackPiece.setFill(Color.BLACK);
            blackPiece.setStroke(Color.DARKGRAY);
            cell.setGraphic(blackPiece);
            break;
        case WHITE:
            Circle whitePiece = new Circle(12);
            whitePiece.setFill(Color.WHITE);
            whitePiece.setStroke(Color.BLACK);
            cell.setGraphic(whitePiece);
            break;
        case BARRIER:
            Rectangle barrier = new Rectangle(20, 20);
            barrier.setFill(Color.SADDLEBROWN);
            cell.setGraphic(barrier);
            break;
        case CRATER:
            StackPane craterGraphic = createCraterEffect();
            cell.setGraphic(craterGraphic);
            break;
    }

    cell.setOnAction(e -> handleCellClick(row, col));
    return cell;
}
```

游戏状态管理

```
private void saveGameState() {
    try {
        gameState = new GameState();
        gameState.saveState(boardManager, boardFinished, player1, player2, showHints);
        gameState.saveToFile(SAVE_FILE);
    } catch (Exception e) {
        System.err.println("保存游戏状态失败: " + e.getMessage());
    }
}

private void loadGameState() {
    try {
        File saveFile = new File(SAVE_FILE);
        if (saveFile.exists()) {
            gameState = new GameState();
            gameState.loadFromFile(SAVE_FILE);
            gameState.restoreState(this);
            updateDisplay();
        }
    } catch (Exception e) {
        System.err.println("加载游戏状态失败: " + e.getMessage());
    }
}
```

事件处理机制

```
private void handleCellClick(int row, int col) {  
    // 游戏状态验证  
    if (boardFinished.get(boardManager.getCurrentBoardIndex() - 1)) {  
        showAlert("游戏已结束", "当前游戏已经结束，请选择其他游戏或创建新游戏。");  
        return;  
    }  
  
    Player currentPlayer = boardManager.getCurrentPlayer();  
    Board currentBoard = boardManager.getCurrentBoard();  
    GameMode currentMode = boardManager.getCurrentMode();  
  
    // 处理特殊模式（炸弹模式）  
    if (bombMode && currentMode instanceof GomokuMode) {  
        handleBombMove(row, col, currentPlayer, currentBoard, currentMode);  
    } else {  
        // 处理常规移动  
        handleNormalMove(row, col, currentPlayer, currentBoard, currentMode);  
    }  
}
```

项目特色

用户体验

- **直观操作**：鼠标点击操作，符合现代应用习惯
- **实时反馈**：即时的游戏状态更新和提示信息
- **错误处理**：友好的错误提示和异常处理

技术亮点

- **模块化设计**：清晰的组件分离和职责划分
- **状态持久化**：完整的游戏状态序列化机制
- **扩展性**：易于添加新的游戏模式和功能

性能优化

- **事件驱动**：高效的JavaFX事件处理机制
- **按需渲染**：智能的界面更新策略

- **内存管理**: 合理的对象生命周期管理

未来改进方向

1. **性能优化**: 优化大量棋盘重绘时的性能
2. **动画效果**: 添加棋子落下、翻转等动画效果
3. **主题定制**: 支持多种界面主题和颜色方案
4. **多语言支持**: 实现国际化的多语言界面
5. **网络对战**: 支持在线多人游戏功能

架构扩展分析：添加2048游戏类型

2048游戏特性分析

2048游戏与现有的棋类游戏有显著差异，主要特点包括：

游戏机制差异

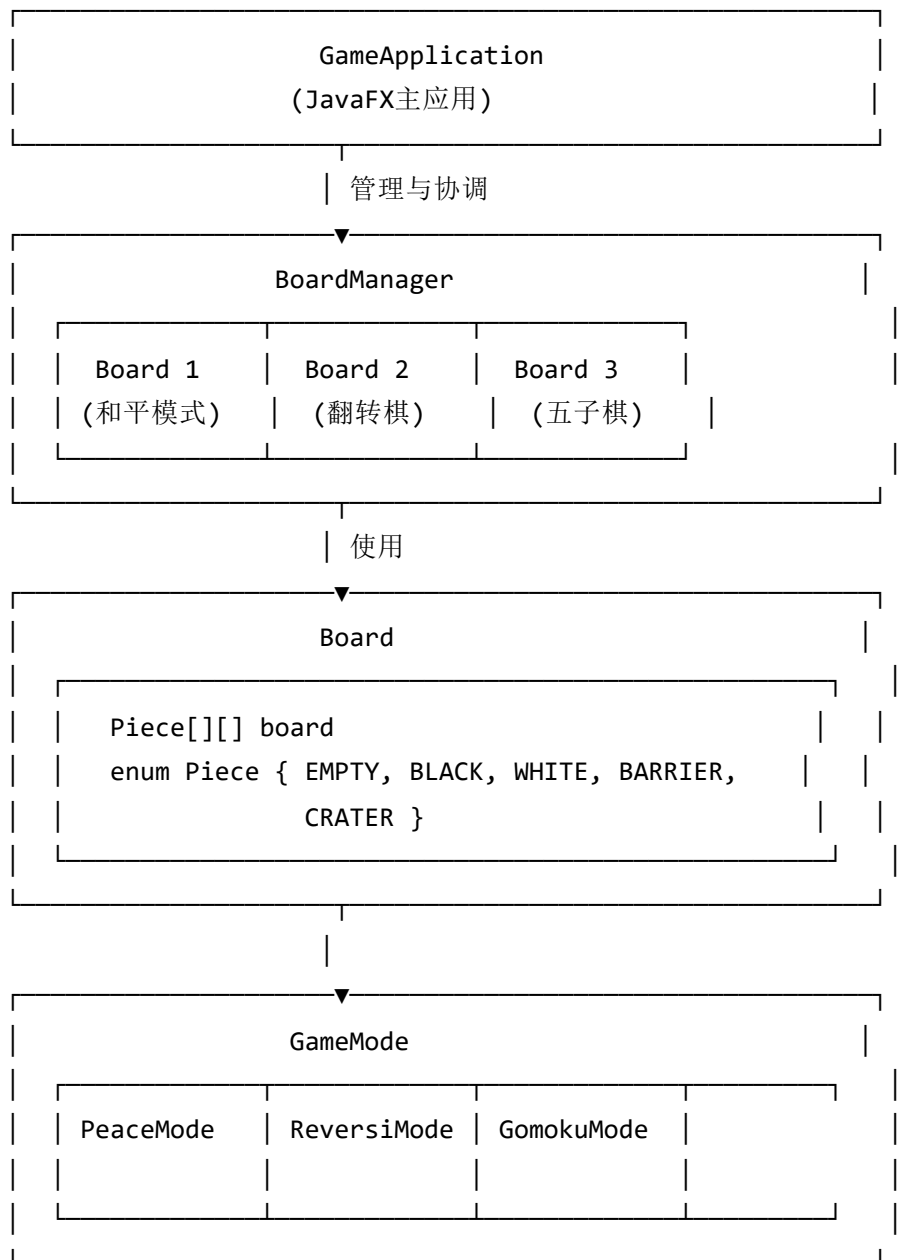
- **操作方式**: 键盘方向键滑动 vs 鼠标点击落子
- **游戏元素**: 数字瓦片(2,4,8,16...) vs 黑白棋子
- **游戏逻辑**: 瓦片合并与移动 vs 棋子放置
- **棋盘规格**: 4x4固定网格 vs 8x8/15x15可变网格
- **胜利条件**: 达到2048瓦片 vs 棋子数量/连线

技术挑战

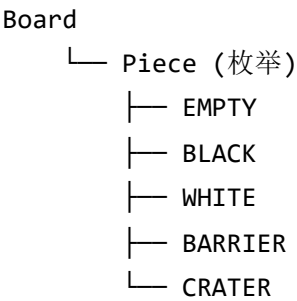
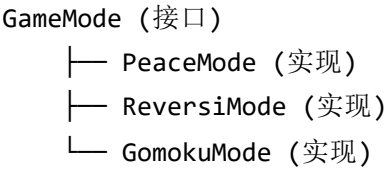
- **输入处理**: 需要键盘事件监听机制
- **动画效果**: 瓦片滑动和合并动画
- **状态管理**: 瓦片值的序列化存储
- **UI渲染**: 数字显示与颜色渐变

现有架构分析

当前系统架构图



类依赖关系图



添加2048游戏所需修改

1. 核心数据结构扩展

Board类扩展：


```

public class Board {
    // 现有的Piece枚举需要扩展或创建新的数据类型
    public enum Piece { EMPTY, BLACK, WHITE, BARRIER, CRATER }

    // 新增：支持数字瓦片的数据结构
    private int[][] numberBoard; // 2048游戏的数字网格
    private boolean is2048Mode = false; // 标识当前是否为2048模式

    // 新增方法
    public void initialize2048Board() {
        numberBoard = new int[4][4];
        is2048Mode = true;
        // 初始化两个随机位置的2
        addRandomTile();
        addRandomTile();
    }

    public int getNumber(int row, int col) { ... }
    public void setNumber(int row, int col, int value) { ... }
    public boolean is2048Mode() { return is2048Mode; }
    public void addRandomTile() { ... } // 随机添加2或4
}

```

新的GameMode实现：

```

public class Game2048Mode implements GameMode {
    @Override
    public boolean isValidMove(Board board, int row, int col, Board.Piece piece) {
        // 2048游戏不使用传统的isValidMove，返回false或重新设计接口
        return false;
    }

    // 新增2048特有的方法
    public boolean canMove(Board board) { ... }
    public boolean moveLeft(Board board) { ... }
    public boolean moveRight(Board board) { ... }
    public boolean moveUp(Board board) { ... }
    public boolean moveDown(Board board) { ... }
    public boolean isWin(Board board) { ... } // 检查是否达到2048
    public int getScore(Board board) { ... } // 计算当前分数
}

```

2. 界面处理机制扩展

GameApplication键盘事件处理：

```
public class GameApplication extends Application {
    // 新增键盘事件处理
    private void setupKeyboardControls(Scene scene) {
        scene.setOnKeyPressed(event -> {
            if (boardManager.getCurrentMode() instanceof Game2048Mode) {
                handle2048KeyPress(event.getCode());
            }
        });
    }

    private void handle2048KeyPress(KeyCode keyCode) {
        Game2048Mode mode = (Game2048Mode) boardManager.getCurrentMode();
        Board board = boardManager.getCurrentBoard();

        boolean moved = false;
        switch (keyCode) {
            case UP: moved = mode.moveUp(board); break;
            case DOWN: moved = mode.moveDown(board); break;
            case LEFT: moved = mode.moveLeft(board); break;
            case RIGHT: moved = mode.moveRight(board); break;
        }

        if (moved) {
            board.addRandomTile();
            updateDisplay();
            check2048WinCondition();
        }
    }
}
```

渲染系统扩展：

```

private Button create2048Cell(int row, int col) {
    Button cell = new Button();
    cell.setPrefSize(60, 60); // 2048需要更大的格子显示数字

    Board currentBoard = boardManager.getCurrentBoard();
    if (currentBoard.is2048Mode()) {
        int number = currentBoard.getNumber(row, col);
        if (number > 0) {
            cell.setText(String.valueOf(number));
            cell.setStyle(get2048CellStyle(number)); // 根据数字设置颜色
        } else {
            cell.setText("");
            cell.setStyle("-fx-background-color: lightgray;");
        }
    }

    // 2048不需要点击事件，或者设置为显示帮助信息
    cell.setOnAction(e -> show2048Help());
    return cell;
}

private String get2048CellStyle(int number) {
    // 根据数字返回不同的颜色样式
    switch (number) {
        case 2: return "-fx-background-color: #eee4da; -fx-text-fill: #776e65;";
        case 4: return "-fx-background-color: #ede0c8; -fx-text-fill: #776e65;";
        case 8: return "-fx-background-color: #f2b179; -fx-text-fill: #f9f6f2;";
        // ... 更多颜色定义
        default: return "-fx-background-color: #3c3a32; -fx-text-fill: #f9f6f2;";
    }
}

```

3. 状态管理扩展

GameState序列化扩展：

```

public class GameState implements Serializable {
    // 现有字段...

    // 新增2048相关状态
    private int[][][] numberBoards; // 存储所有2048棋盘的数字状态
    private boolean[] is2048ModeArray; // 标识哪些棋盘是2048模式
    private int[] scores2048; // 2048游戏的分数

    public void saveState(BoardManager boardManager, ...) {
        // 现有保存逻辑...

        // 新增：保存2048状态
        save2048State(boardManager);
    }

    private void save2048State(BoardManager boardManager) {
        int boardCount = boardManager.getBoardCount();
        numberBoards = new int[boardCount][][];
        is2048ModeArray = new boolean[boardCount];
        scores2048 = new int[boardCount];

        for (int i = 0; i < boardCount; i++) {
            Board board = boardManager.getBoard(i);
            if (board.is2048Mode()) {
                is2048ModeArray[i] = true;
                numberBoards[i] = board.getNumberBoard(); // 需要新增此方法
                scores2048[i] = ((Game2048Mode)boardManager.getMode(i)).getScore(board);
            }
        }
    }
}

```

4. 用户界面调整

信息显示面板：

```

// 在createPlayerInfoColumn()中新增2048相关信息显示
private Label scoreLabel2048;
private Label bestTileLabel;
private Label movesCountLabel;

private void updateInfoDisplay() {
    // 现有更新逻辑...

    // 新增：2048信息更新
    if (boardManager.getCurrentBoard().is2048Mode()) {
        Game2048Mode mode = (Game2048Mode) boardManager.getCurrentMode();
        scoreLabel2048.setText("分数: " + mode.getScore(boardManager.getCurrentBoard()));
        bestTileLabel.setText("最大瓦片: " + mode.getBestTile(boardManager.getCurrentBoard()));
        movesCountLabel.setText("移动次数: " + mode.getMoveCount());

        // 隐藏棋类游戏相关信息
        blackPlayerLabel.setVisible(false);
        whitePlayerLabel.setVisible(false);
        blackScoreLabel.setVisible(false);
        whiteScoreLabel.setVisible(false);
    } else {
        // 显示棋类游戏信息，隐藏2048信息
        scoreLabel2048.setVisible(false);
        bestTileLabel.setVisible(false);
        movesCountLabel.setVisible(false);
    }
}
}

```

控制按钮调整：

```
private Button new2048Button;
private Button restart2048Button;

private VBox createControlsColumn() {
    // 现有按钮...

    // 新增2048相关按钮
    new2048Button = new Button("新建2048");
    new2048Button.setOnAction(e -> createNew2048Game());

    restart2048Button = new Button("重启2048");
    restart2048Button.setOnAction(e -> restart2048Game());

    // 添加到控制面板
    column.getChildren().addAll(/* 现有按钮 */, new2048Button, restart2048Button);

    return column;
}

private void updateButtonStates() {
    // 现有按钮状态更新...

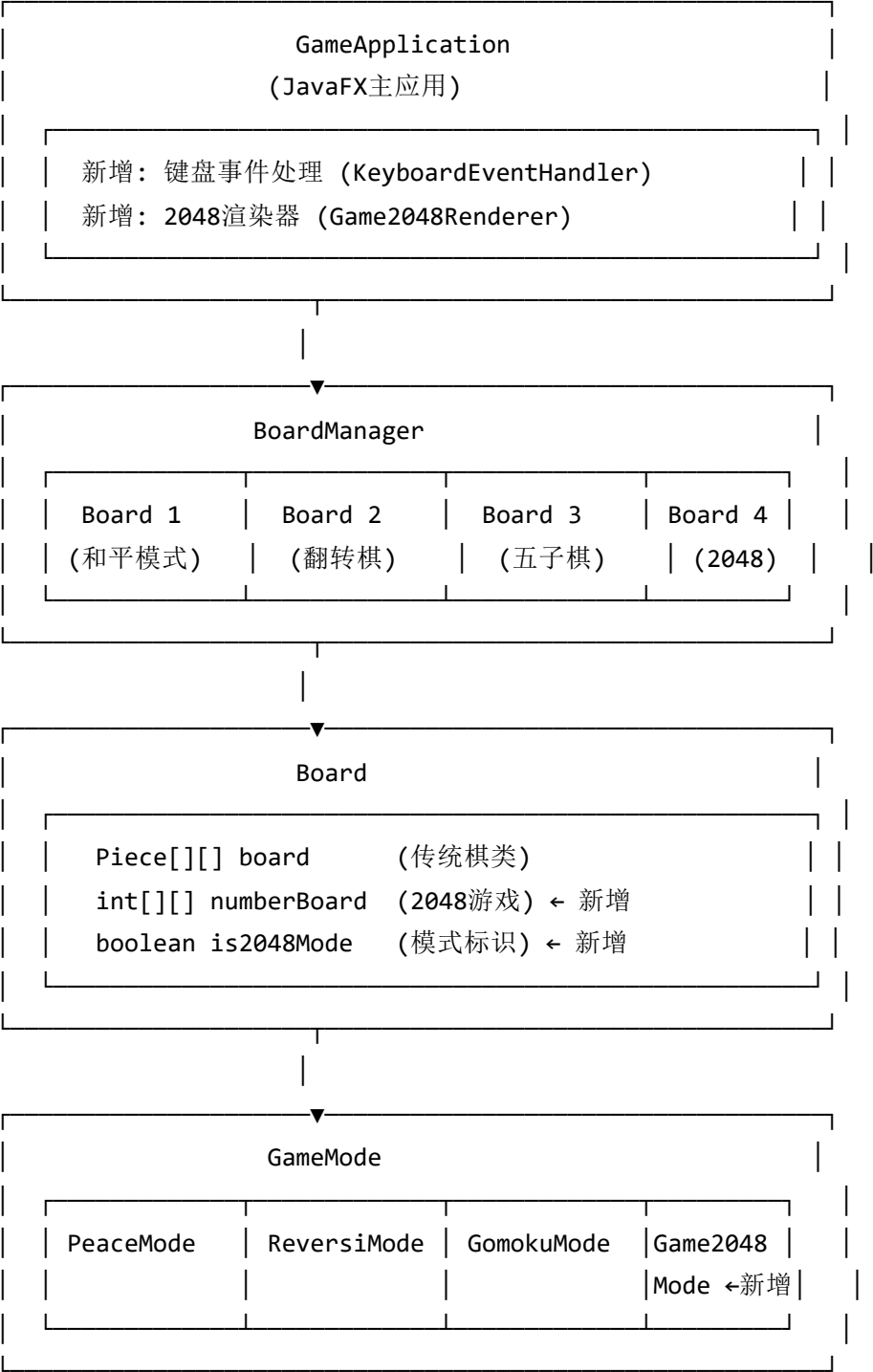
    boolean is2048 = boardManager.getCurrentBoard().is2048Mode();

    // 2048模式下隐藏棋类游戏按钮
    passButton.setVisible(!is2048);
    bombButton.setVisible(!is2048);
    hintButton.setVisible(!is2048);

    // 显示2048特有按钮
    restart2048Button.setVisible(is2048);
    new2048Button.setVisible(true); // 始终显示，用于创建新的2048游戏
}
```

扩展后的系统架构

更新后的架构图



新增组件关系图

Game2048Mode

- └─ 实现 GameMode 接口
- └─ 依赖 Board (numberBoard)
- └─ 新增方法：
 - └─ moveLeft/Right/Up/Down()
 - └─ canMove()
 - └─ isWin()
 - └─ getScore()

Board

- └─ 扩展数据结构：
 - └─ int[][] numberBoard
 - └─ boolean is2048Mode
- └─ 新增方法：
 - └─ initialize2048Board()
 - └─ getNumber()/setNumber()
 - └─ addRandomTile()
 - └─ getNumberBoard()

GameApplication

- └─ 新增键盘事件处理
- └─ 新增2048渲染逻辑
- └─ 新增2048专用UI组件
- └─ 修改信息显示逻辑

GameState

- └─ 扩展序列化字段：
 - └─ int[][][] numberBoards
 - └─ boolean[] is2048ModeArray
 - └─ int[] scores2048
- └─ 新增序列化方法



架构设计优势

通过这种扩展方式，我们可以验证现有架构的以下优势：

1. **良好的扩展性**：GameMode接口设计使得添加新游戏类型相对容易
2. **模块化设计**：各组件职责清晰，修改影响范围可控
3. **状态管理**：现有的序列化机制可以方便地扩展支持新数据类型
4. **UI框架**：JavaFX架构支持灵活的事件处理和界面定制

这个扩展案例充分展示了良好软件架构在面对需求变更时的适应能力和扩展潜力。